

D4_Deep_Learning_Xarxes_Neuronals_avançades_i_models_preentrenats

February 21, 2026

1 D4. Deep Learning: Xarxes Neuronals avançades i models preentrenats

Les xarxes neuronals van més enllà de les MLP (xarxes multilayer perceptron, ideals per classificació simple) i les CNN (convolucionals, perfectes per imatges), amb variants especialitzades per problemes complexos com seqüències temporals, generació de dades o decisions autònomes.

Hi ha diverses famílies adaptades a dades estructurades diferents.

- **RNN (Recurrent Neural Networks):** Processen seqüències mantenint memòria de passos anteriors. Molt utilitzades en text i series temporals.
- **GAN (Generative Adversarial Networks):** Dues xarxes competint (generador vs. discriminador) per crear dades realistes, com imatges sintètiques.
- **Transformers:** Basats en atenció (parts importants), revolucionen el NLP. no seqüencials estrictes, sinó paral·lels. Exemple: models com BERT per traducció.

No entrarem en detall en aquestes xarxes neuronals tant complexes i segurament mai n'entrenarem una des de zero.

##Hugging Face

Hugging Face és la plataforma de referència per a models d'IA de codi obert. Funciona com un registre centralitzat on investigadors i empreses publiquen els seus models amb els pesos preentrenats, documentació i codi d'exemple.

<https://huggingface.co/>

Components principals:

- **Model Hub:** +500k models publicats (Google, Meta, Microsoft, etc.)
- **Datasets Hub:** +75k datasets
- **Spaces:** demos interactives amb Gradio o Streamlit
- **Transformers library:** API unificada per a inferència i fine-tuning

1.0.1 Concepte de *Transfer Learning*

Entrenar models com ViT-Large o LLaMA-70B des de zero requereix recursos computacionals massius i temps prohibitivament llargs, però podem aprofitar models preentrenats mitjançant transfer learning.

Es carreguen els pesos preentrenats (que ja contenen coneixements generals sobre imatges o llenguatge), es reemplaça la capa de sortida per una adaptada a la nostra tasca específica i es realitza *fine-tuning* amb les nostres dades.

El resultat és un model que assoleix un rendiment excepcional amb molt menys temps d'entrenament i dades, cosa completament inviable si s'entrenés des de zero.

###Exemple d'ús de models preentrenats

El pipeline gestiona internament la descàrrega del model, el preprocessament de l'entrada, la inferència, i la conversió dels *logits* a etiquetes llegibles.

```
[ ]: from transformers import pipeline

clf = pipeline(
    task = "image-classification",
    model = "google/vit-base-patch16-224"
)

url_imatge = "https://www.infocampo.com.ar/wp-content/uploads/2019/09/durian.
↪jpg"

resultats = clf(url_imatge)

for r in resultats[:5]:
    print(f"{r['label']:25} {r['score']:.1%}")
```

```
Loading weights: 0%|          | 0/200 [00:00<?, ?it/s]
```

```
Fast image processor class <class
'transformers.models.vit.image_processing_vit_fast.ViTImageProcessorFast'> is
available for this model. Using slow image processor class. To use the fast
image processor class set `use_fast=True`.
```

```
jackfruit, jak, jack      99.4%
buckeye, horse chestnut, conker 0.2%
custard apple             0.1%
apiary, bee house         0.0%
pineapple, ananas         0.0%
```

```
[ ]: import matplotlib.pyplot as plt
from PIL import Image
import requests
plt.imshow(Image.open(requests.get(url_imatge, stream=True).raw))
```

```
[ ]: <matplotlib.image.AxesImage at 0x7be4d0c7a6c0>
```



Exemple de transfer learning Utilitzarem el dataset Beans com a exemple a *transfer learning* i *fine-tuning*, ja que agafem un model pre-entrenat (ViT-base) i l'adaptem a la detecció de malalties de plantes.

El dataset Beans conté imatges de fulles de mongeta etiquetades en 3 classes: planta sana i dues malalties (*angular leaf spot* i *bean rust*).

```
[ ]: from datasets import load_dataset

beans = load_dataset('beans')
classes = beans['train'].features['labels'].names
```

Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.
 WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faster downloads.

```
[ ]: print(classes)

['angular_leaf_spot', 'bean_rust', 'healthy']
```

```
[ ]: print(f'Particions: train={len(beans["train"])} | val={len(beans["validation"])} | test={len(beans["test"])}')
```

Particions: train=1034 | val=133 | test=128

La divisió train/validation/test és un estàndard per garantir models robustos:

- **train** : Entrenar el model (ajustar pesos) (70-80%)

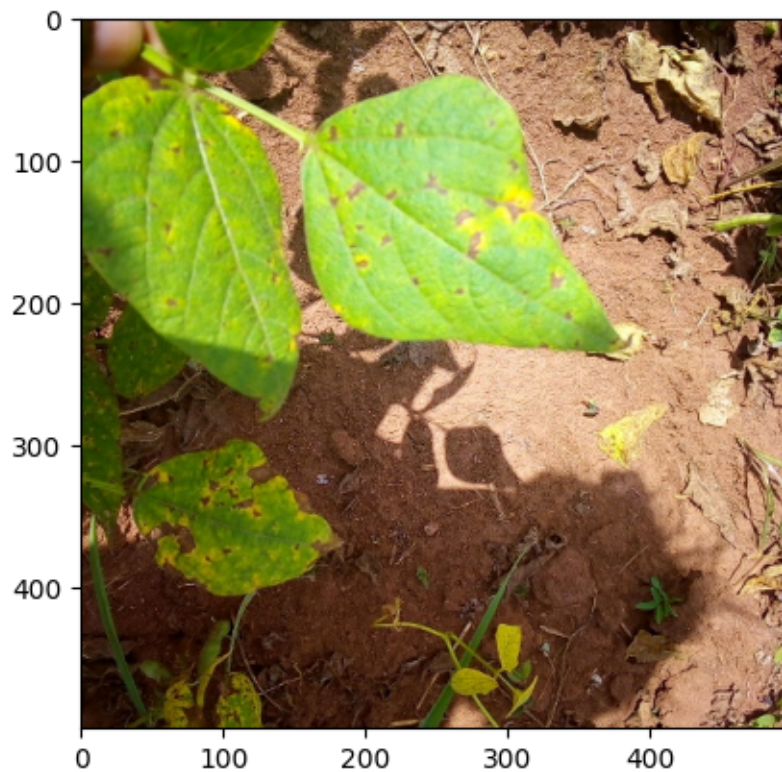
- **validation** : Ajustar hiperparàmetres, triar millor model (10-15%)
- **test** : Avaluar rendiment final (estimar generalització) 10-15%

```
[ ]: print(beans['train'].features['labels'])
```

```
ClassLabel(names=['angular_leaf_spot', 'bean_rust', 'healthy'])
```

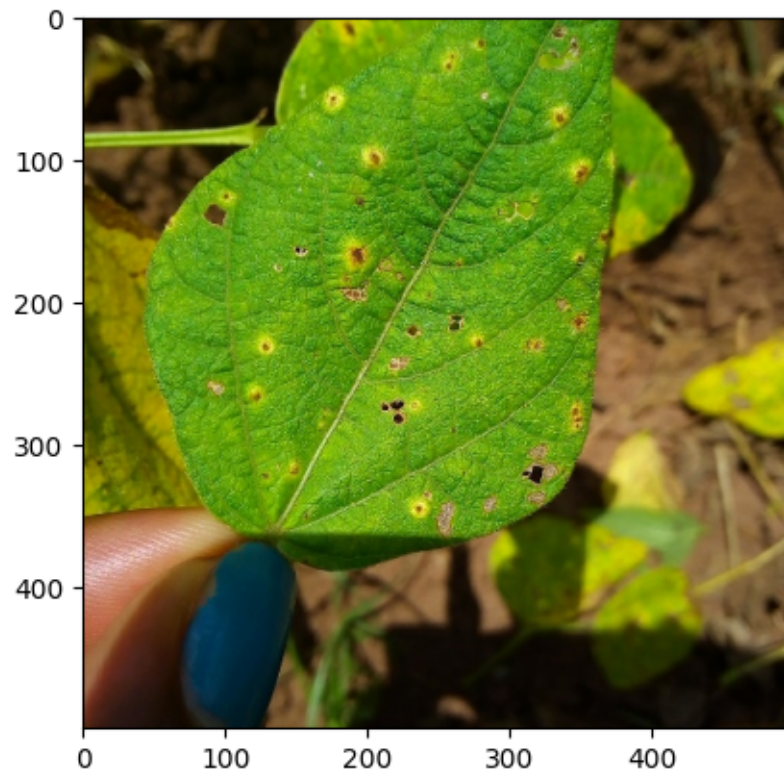
```
[ ]: plt.imshow(beans['train'][0]['image'])  
print(beans['train'][0]['labels'])
```

0



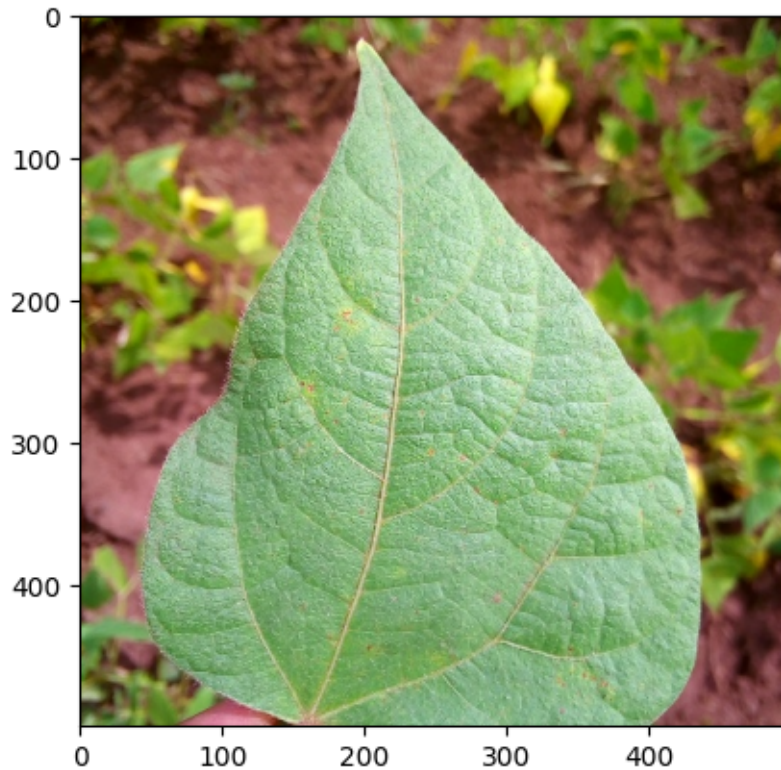
```
[ ]: plt.imshow(beans['train'][500]['image'])  
print(beans['train'][500]['labels'])
```

1



```
[ ]: plt.imshow(beans['train'][800]['image'])  
      print(beans['train'][800]['labels'])
```

2



El **fine-tuning** consisteix a agafar un model que ja ha estat entrenat amb milions d'imatges i re-entrenar-lo amb les nostres dades perquè aprengui les classes que ens interessen. El model ja “sap veure”: només li hem d'ensenyar a identificar les nostres categories concretes.

El procés té 4 passos:

- Carregar el model preentrenat
- Preparar les imatges (mida i normalització estàndard del model)
- Configurar l'entrenament
- Entrenar i avaluar

```
[ ]: from datasets import load_dataset
from transformers import AutoImageProcessor, AutoModelForImageClassification, \
    Trainer, TrainingArguments, EvalPrediction
from sklearn.metrics import accuracy_score
import numpy as np

beans = load_dataset('beans')
classes = beans['train'].features['labels'].names

processor = AutoImageProcessor.from_pretrained('google/vit-base-patch16-224')

def preparar_imatges(exemples):
```

```

    imatges = [img.convert('RGB') for img in examples['image']]
    encoding = processor(imatges, return_tensors='pt')
    encoding['labels'] = examples['labels']
    return encoding

train_ds = beans['train'].with_transform(preparar_imatges)
val_ds = beans['validation'].with_transform(preparar_imatges)

model = AutoModelForImageClassification.from_pretrained(
    'google/vit-base-patch16-224',
    num_labels=len(classes),
    ignore_mismatched_sizes=True
)

def compute_metrics(eval_pred: EvalPrediction):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    return {"accuracy": accuracy_score(labels, predictions)}

trainer = Trainer(
    model=model,
    train_dataset=train_ds,
    eval_dataset=val_ds,
    compute_metrics=compute_metrics,
    args=TrainingArguments(
        output_dir='./beans-vit-model',
        num_train_epochs=2, #MOLT BAIX
        per_device_train_batch_size=2, # MOLT BAIX
        eval_strategy='epoch',
        save_strategy='epoch',
        remove_unused_columns=False,
    )
)

trainer.train()

trainer.save_model('./beans-model-final')
processor.save_pretrained('./beans-model-final')

```

Fast image processor class <class 'transformers.models.vit.image_processing_vit_fast.ViTImageProcessorFast'> is available for this model. Using slow image processor class. To use the fast image processor class set `use\_fast=True`.

Loading weights: 0% | 0/200 [00:00<?, ?it/s]

ViTForImageClassification LOAD REPORT from: google/vit-base-patch16-224

Key	Status
-----	--------

-----+-----+-----

```
-----
classifier.bias | MISMATCH | Reinit due to size mismatch ckpt:
torch.Size([1000]) vs model:torch.Size([3])
classifier.weight | MISMATCH | Reinit due to size mismatch ckpt:
torch.Size([1000, 768]) vs model:torch.Size([3, 768])
```

Notes:

- MISMATCH :ckpt weights were loaded, but they did not match the original empty weight shapes.

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775:

UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.

```
super().__init__(loader)
```

<IPython.core.display.HTML object>

Writing model shards: 0% | 0/1 [00:00<?, ?it/s]

/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775:

UserWarning: 'pin_memory' argument is set as true but no accelerator is found, then device pinned memory won't be used.

```
super().__init__(loader)
```

```
-----
KeyboardInterrupt                                Traceback (most recent call last)
/tmp/ipython-input-2531192078.py in <cell line: 0>()
    44 )
    45
--> 46 trainer.train()
    47
    48 trainer.save_model('./beans-model-final')

/usr/local/lib/python3.12/dist-packages/transformers/trainer.py in train(self,
    ↪ resume_from_checkpoint, trial, ignore_keys_for_eval)
   2172         hf_hub_utils.enable_progressBars()
   2173     else:
-> 2174         return inner_training_loop(
   2175             args=args,
   2176             resume_from_checkpoint=resume_from_checkpoint,

/usr/local/lib/python3.12/dist-packages/transformers/trainer.py in
    ↪ _inner_training_loop(self, batch_size, args, resume_from_checkpoint, trial,
    ↪ ignore_keys_for_eval)
   2534         )
   2535         with context():
-> 2536             tr_loss_step = self.training_step(model, inputs
    ↪ num_items_in_batch)
   2537
   2538         if (
```

```

/usr/local/lib/python3.12/dist-packages/transformers/trainer.py in
↳ training_step(self, model, inputs, num_items_in_batch)
    3807
    3808         with self.compute_loss_context_manager():
-> 3809             loss = self.compute_loss(model, inputs,
↳ num_items_in_batch=num_items_in_batch)
    3810
    3811         del inputs

/usr/local/lib/python3.12/dist-packages/transformers/trainer.py in
↳ compute_loss(self, model, inputs, return_outputs, num_items_in_batch)
    3878             kwargs["num_items_in_batch"] = num_items_in_batch
    3879             inputs = {**inputs, **kwargs}
-> 3880             outputs = model(**inputs)
    3881
    3882             # User-defined compute_loss function

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py in
↳ _wrapped_call_impl(self, *args, **kwargs)
    1774             return self._compiled_call_impl(*args, **kwargs) # type:
↳ ignore[misc]
    1775         else:
-> 1776             return self._call_impl(*args, **kwargs)
    1777
    1778         # torchrec tests the code consistency with the following code

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py in
↳ _call_impl(self, *args, **kwargs)
    1785             or _global_backward_pre_hooks or _global_backward_hooks
    1786             or _global_forward_hooks or _global_forward_pre_hooks):
-> 1787             return forward_call(*args, **kwargs)
    1788
    1789             result = None

/usr/local/lib/python3.12/dist-packages/transformers/utils/generic.py in
↳ wrapper(self, *args, **kwargs)
    833         if return_dict_passed is not None:
    834             return_dict = return_dict_passed
--> 835         output = func(self, *args, **kwargs)
    836         if not return_dict and not isinstance(output, tuple):
    837             output = output.to_tuple()

/usr/local/lib/python3.12/dist-packages/transformers/models/vit/modeling_vit.py
↳ in forward(self, pixel_values, labels, interpolate_pos_encoding, **kwargs)
    614         """
    615

```

```

--> 616         outputs: BaseModelOutputWithPooling = self.vit(
        617             pixel_values,
        618             interpolate_pos_encoding=interpolate_pos_encoding,

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py in
↳ _wrapped_call_impl(self, *args, **kwargs)
    1774         return self._compiled_call_impl(*args, **kwargs) # type:
↳ ignore[misc]
    1775     else:
-> 1776         return self._call_impl(*args, **kwargs)
    1777
    1778     # torchrec tests the code consistency with the following code

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py in
↳ _call_impl(self, *args, **kwargs)
    1785         or _global_backward_pre_hooks or _global_backward_hooks
    1786         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1787         return forward_call(*args, **kwargs)
    1788
    1789         result = None

/usr/local/lib/python3.12/dist-packages/transformers/utils/generic.py in
↳ wrapper(self, *args, **kwargs)
    1000             outputs = func(self, *args, **kwargs)
    1001         else:
-> 1002             outputs = func(self, *args, **kwargs)
    1003         except TypeError as original_exception:
    1004             # If we get a TypeError, it's possible that the model is
↳ not receiving the recordable kwargs correctly.

/usr/local/lib/python3.12/dist-packages/transformers/models/vit/modeling_vit.py
↳ in forward(self, pixel_values, bool_masked_pos, interpolate_pos_encoding,
↳ **kwargs)
    432     )
    433
--> 434     encoder_outputs: BaseModelOutput = self.encoder(embedding_output)
    435
    436     sequence_output = encoder_outputs.last_hidden_state

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py in
↳ _wrapped_call_impl(self, *args, **kwargs)
    1774         return self._compiled_call_impl(*args, **kwargs) # type:
↳ ignore[misc]
    1775     else:
-> 1776         return self._call_impl(*args, **kwargs)
    1777
    1778     # torchrec tests the code consistency with the following code

```

```

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py in
↳ _call_impl(self, *args, **kwargs)
    1785         or _global_backward_pre_hooks or _global_backward_hooks
    1786         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1787         return forward_call(*args, **kwargs)
    1788
    1789         result = None

/usr/local/lib/python3.12/dist-packages/transformers/models/vit/modeling_vit.py
↳ in forward(self, hidden_states)
    343     def forward(self, hidden_states: torch.Tensor) -> BaseModelOutput:
    344         for i, layer_module in enumerate(self.layer):
--> 345             hidden_states = layer_module(hidden_states)
    346
    347         return BaseModelOutput(last_hidden_state=hidden_states)

/usr/local/lib/python3.12/dist-packages/transformers/modeling_layers.py in
↳ __call__(self, *args, **kwargs)
    91
    92         return self._gradient_checkpointing_func(partial(super().
↳ __call__, **kwargs), *args)
---> 93         return super().__call__(*args, **kwargs)
    94
    95

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py in
↳ _wrapped_call_impl(self, *args, **kwargs)
    1774         return self._compiled_call_impl(*args, **kwargs) # type:
↳ ignore[misc]
    1775     else:
-> 1776         return self._call_impl(*args, **kwargs)
    1777
    1778     # torchrec tests the code consistency with the following code

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py in
↳ _call_impl(self, *args, **kwargs)
    1785         or _global_backward_pre_hooks or _global_backward_hooks
    1786         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1787         return forward_call(*args, **kwargs)
    1788
    1789         result = None

/usr/local/lib/python3.12/dist-packages/transformers/models/vit/modeling_vit.py
↳ in forward(self, hidden_states)
    329
    330     # second residual connection is done here
--> 331     layer_output = self.output(layer_output, hidden_states)

```

```

332
333         return layer_output

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py in
↳ _wrapped_call_impl(self, *args, **kwargs)
1774         return self._compiled_call_impl(*args, **kwargs) # type:
↳ ignore[misc]
1775     else:
-> 1776         return self._call_impl(*args, **kwargs)
1777
1778     # torchrec tests the code consistency with the following code

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py in
↳ _call_impl(self, *args, **kwargs)
1785         or _global_backward_pre_hooks or _global_backward_hooks
1786         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1787         return forward_call(*args, **kwargs)
1788
1789         result = None

/usr/local/lib/python3.12/dist-packages/transformers/models/vit/modeling_vit.py
↳ in forward(self, hidden_states, input_tensor)
298
299     def forward(self, hidden_states: torch.Tensor, input_tensor: torch.
↳ Tensor) -> torch.Tensor:
--> 300         hidden_states = self.dense(hidden_states)
301         hidden_states = self.dropout(hidden_states)
302         hidden_states = hidden_states + input_tensor

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py in
↳ _wrapped_call_impl(self, *args, **kwargs)
1774         return self._compiled_call_impl(*args, **kwargs) # type:
↳ ignore[misc]
1775     else:
-> 1776         return self._call_impl(*args, **kwargs)
1777
1778     # torchrec tests the code consistency with the following code

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py in
↳ _call_impl(self, *args, **kwargs)
1785         or _global_backward_pre_hooks or _global_backward_hooks
1786         or _global_forward_hooks or _global_forward_pre_hooks):
-> 1787         return forward_call(*args, **kwargs)
1788
1789         result = None

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/linear.py in
↳ forward(self, input)

```

```

132         Runs the forward pass.
133         """
--> 134         return F.linear(input, self.weight, self.bias)
135
136     def extra_repr(self) -> str:

```

KeyboardInterrupt:

##Altres tasques amb Hugging Face

Disposen d'infinitat de models disponibles no tant sols enfocats a la classificació d'imatge sinó a moltes altres tasques:

###Anàlisi de sentiment

```

[ ]: from transformers import pipeline

# Model multilingüe (català, castellà, anglès...)
sentiment = pipeline(
    'sentiment-analysis',
    model='nlptown/bert-base-multilingual-uncased-sentiment'
)

comentarios = [
    'Python és el millor llenguatge del Python',
    'La professora ens abandona en 3-4 classes',
    'Tot és culpa del overfitting'
]

for c in comentarios:
    result = sentiment(c)[0]
    label = "POSITIU" if result['label'] in ['4 stars', '5 stars'] else
    ↪ "NEGATIU"
    print(f"{label:<12} {result['score']:.0%} | {c}")

```

Loading weights: 0% | 0/201 [00:00<?, ?it/s]

```

POSITIU      83% | Python és el millor llenguatge del Python
NEGATIU      29% | La professora ens abandona en 3-4 classes
NEGATIU      49% | Tot és culpa del overfitting

```

###Resum de texts

```

[ ]: from transformers import AutoTokenizer, AutoModelForSeq2SeqLM
import torch

# Càrrega manual del model BART
tokenizer = AutoTokenizer.from_pretrained("facebook/bart-large-cnn")
model = AutoModelForSeq2SeqLM.from_pretrained("facebook/bart-large-cnn")

```

```

article = """Barack Obama va acceptar respondre una ronda de preguntes ràpides
↳ en una entrevista al pòdcast del periodista Brian Tyler Cohen, emesa
↳ dissabte passat.
A la pregunta sobre si els extraterrestres són reals, Obama va afirmar: "Són
↳ reals, però jo no els he vist, i no els tenen a... l'Àrea 51. No hi ha cap
↳ instal·lació subterrània tret que hi hagi aquesta enorme conspiració i
↳ l'hagin amagat al president dels Estats Units", va aclarir Obama. I en to
↳ distès va respondre que la seva primera pregunta quan va prendre possessió a
↳ la Casa Blanca va ser: "On són els àliens?"
Posteriorment, l'expresident Obama ha precisat a Instagram que durant la seva
↳ presidència no va veure "proves" de contacte amb extraterrestres, encara que
↳ va assenyalar que, atès el vast de l'univers, és probable que hi hagi vida
↳ més enllà de la Terra, però les enormes distàncies fan poc probable que ens
↳ hagin visitat."""

# Tokenitza + genera resum
inputs = tokenizer(article, return_tensors="pt", max_length=1024,
↳ truncation=True)
summary_ids = model.generate(
    inputs.input_ids,
    max_length=70,
    min_length=30,
    length_penalty=2.0,
    num_beams=4,
    do_sample=False
)

resum = tokenizer.decode(summary_ids[0], skip_special_tokens=True)

print(f'Original: {len(article.split())} paraules')
print(f'Resum: {len(resum.split())} paraules\n')
print(resum)

```

Loading weights: 0% | 0/511 [00:00<?, ?it/s]

Original: 151 paraules

Resum: 29 paraules

Barack Obama va acceptar respondre una ronda de preguntes ràpides en una entrevista. "No hi ha cap instal·lació subterrània tret that hi hagi aquesta enorme conspiració," va aclarir Obama.

###Detecció d'Objectes

```

[ ]: detector = pipeline('object-detection') # sino ho especifiquem agafa el model
↳ per defecte

```

```

detections = detector('https://huggingface.co/datasets/huggingface/
↳documentation-images/resolve/main/pipeline-cat-chonk.jpeg')

print(f'{len(detections)} objectes:')
for d in detections: print(f'  {d["score"]:.0%} {d["label"]}')

```

No model was supplied, defaulted to facebook/detr-resnet-50 and revision 1d5f47b.

Using a pipeline without specifying a model name and revision in production is not recommended.

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:

UserWarning: for conv1.weight: copying from a non-meta parameter in the checkpoint to a meta parameter in the current model, which is a no-op. (Did you mean to pass `assign=True` to assign items in the state dictionary to their corresponding key in the module instead of copying them in place?)

module._load_from_state_dict(

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:

UserWarning: for bn1.weight: copying from a non-meta parameter in the checkpoint to a meta parameter in the current model, which is a no-op. (Did you mean to pass `assign=True` to assign items in the state dictionary to their corresponding key in the module instead of copying them in place?)

super()._load_from_state_dict(

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:

UserWarning: for bn1.bias: copying from a non-meta parameter in the checkpoint to a meta parameter in the current model, which is a no-op. (Did you mean to pass `assign=True` to assign items in the state dictionary to their corresponding key in the module instead of copying them in place?)

super()._load_from_state_dict(

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:

UserWarning: for bn1.running_mean: copying from a non-meta parameter in the checkpoint to a meta parameter in the current model, which is a no-op. (Did you mean to pass `assign=True` to assign items in the state dictionary to their corresponding key in the module instead of copying them in place?)

super()._load_from_state_dict(

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:

UserWarning: for bn1.running_var: copying from a non-meta parameter in the checkpoint to a meta parameter in the current model, which is a no-op. (Did you mean to pass `assign=True` to assign items in the state dictionary to their corresponding key in the module instead of copying them in place?)

super()._load_from_state_dict(

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:

UserWarning: for bn1.num_batches_tracked: copying from a non-meta parameter in the checkpoint to a meta parameter in the current model, which is a no-op. (Did you mean to pass `assign=True` to assign items in the state dictionary to their corresponding key in the module instead of copying them in place?)

super()._load_from_state_dict(

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:

UserWarning: for layer1.0.conv1.weight: copying from a non-meta parameter in the

```

checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.bn1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.bn1.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.bn1.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.bn1.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.bn1.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer1.0.conv2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.bn2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:

```

```

UserWarning: for layer1.0.bn2.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.bn2.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.bn2.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.bn2.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer1.0.conv3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.bn3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.bn3.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.bn3.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(

```

```

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.bn3.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.bn3.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer1.0.downsample.0.weight: copying from a non-meta parameter
in the checkpoint to a meta parameter in the current model, which is a no-op.
(Did you mean to pass `assign=True` to assign items in the state dictionary to
their corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.downsample.1.weight: copying from a non-meta parameter
in the checkpoint to a meta parameter in the current model, which is a no-op.
(Did you mean to pass `assign=True` to assign items in the state dictionary to
their corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.downsample.1.bias: copying from a non-meta parameter
in the checkpoint to a meta parameter in the current model, which is a no-op.
(Did you mean to pass `assign=True` to assign items in the state dictionary to
their corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.downsample.1.running_mean: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.downsample.1.running_var: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.0.downsample.1.num_batches_tracked: copying from a non-
meta parameter in the checkpoint to a meta parameter in the current model, which

```

is a no-op. (Did you mean to pass `assign=True` to assign items in the state dictionary to their corresponding key in the module instead of copying them in place?)

```
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer1.1.conv1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.1.bn1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.1.bn1.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.1.bn1.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.1.bn1.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.1.bn1.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer1.1.conv2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
```

```

UserWarning: for layer1.1.bn2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.1.bn2.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.1.bn2.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.1.bn2.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.1.bn2.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer1.1.conv3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.1.bn3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.1.bn3.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(

```

```

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.1.bn3.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.1.bn3.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.1.bn3.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer1.2.conv1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.2.bn1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.2.bn1.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.2.bn1.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.2.bn1.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)

```

```

    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.2.bn1.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer1.2.conv2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.2.bn2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.2.bn2.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.2.bn2.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.2.bn2.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.2.bn2.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer1.2.conv3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you

```

```

mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.2.bn3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.2.bn3.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.2.bn3.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.2.bn3.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer1.2.bn3.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer2.0.conv1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.bn1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.bn1.bias: copying from a non-meta parameter in the

```

```

checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.bn1.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.bn1.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.bn1.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer2.0.conv2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.bn2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.bn2.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.bn2.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:

```

```

UserWarning: for layer2.0.bn2.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.bn2.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer2.0.conv3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.bn3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.bn3.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.bn3.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.bn3.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.bn3.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)

```

```

    super().__load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer2.0.downsample.0.weight: copying from a non-meta parameter
in the checkpoint to a meta parameter in the current model, which is a no-op.
(Did you mean to pass `assign=True` to assign items in the state dictionary to
their corresponding key in the module instead of copying them in place?)
    module.__load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.downsample.1.weight: copying from a non-meta parameter
in the checkpoint to a meta parameter in the current model, which is a no-op.
(Did you mean to pass `assign=True` to assign items in the state dictionary to
their corresponding key in the module instead of copying them in place?)
    super().__load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.downsample.1.bias: copying from a non-meta parameter
in the checkpoint to a meta parameter in the current model, which is a no-op.
(Did you mean to pass `assign=True` to assign items in the state dictionary to
their corresponding key in the module instead of copying them in place?)
    super().__load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.downsample.1.running_mean: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super().__load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.downsample.1.running_var: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super().__load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.0.downsample.1.num_batches_tracked: copying from a non-
meta parameter in the checkpoint to a meta parameter in the current model, which
is a no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super().__load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer2.1.conv1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module.__load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.1.bn1.weight: copying from a non-meta parameter in the

```

```

checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.1.bn1.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.1.bn1.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.1.bn1.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.1.bn1.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer2.1.conv2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.1.bn2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.1.bn2.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:

```

```

UserWarning: for layer2.1.bn2.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.1.bn2.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.1.bn2.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer2.1.conv3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.1.bn3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.1.bn3.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.1.bn3.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.1.bn3.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(

```

```

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.1.bn3.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer2.2.conv1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.2.bn1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.2.bn1.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.2.bn1.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.2.bn1.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.2.bn1.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer2.2.conv2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their

```

```

corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.2.bn2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.2.bn2.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.2.bn2.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.2.bn2.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.2.bn2.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer2.2.conv3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.2.bn3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.2.bn3.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you

```

```

mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.2.bn3.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.2.bn3.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.2.bn3.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer2.3.conv1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.3.bn1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.3.bn1.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.3.bn1.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.3.bn1.running_var: copying from a non-meta parameter in

```

the checkpoint to a meta parameter in the current model, which is a no-op. (Did you mean to pass `assign=True` to assign items in the state dictionary to their corresponding key in the module instead of copying them in place?)

```
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.3.bn1.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
```

```
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer2.3.conv2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
```

```
module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.3.bn2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
```

```
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.3.bn2.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
```

```
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.3.bn2.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
```

```
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.3.bn2.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
```

```
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.3.bn2.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
```

```
super()._load_from_state_dict(
```

```

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer2.3.conv3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.3.bn3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.3.bn3.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.3.bn3.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.3.bn3.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer2.3.bn3.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.0.conv1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.bn1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)

```

```

    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.bn1.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.bn1.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.bn1.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.bn1.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.0.conv2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.bn2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.bn2.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.bn2.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their

```

```

corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.bn2.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.bn2.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.0.conv3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.bn3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.bn3.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.bn3.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.bn3.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.bn3.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a

```

```

no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.0.downsample.0.weight: copying from a non-meta parameter
in the checkpoint to a meta parameter in the current model, which is a no-op.
(Did you mean to pass `assign=True` to assign items in the state dictionary to
their corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.downsample.1.weight: copying from a non-meta parameter
in the checkpoint to a meta parameter in the current model, which is a no-op.
(Did you mean to pass `assign=True` to assign items in the state dictionary to
their corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.downsample.1.bias: copying from a non-meta parameter
in the checkpoint to a meta parameter in the current model, which is a no-op.
(Did you mean to pass `assign=True` to assign items in the state dictionary to
their corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.downsample.1.running_mean: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.downsample.1.running_var: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.0.downsample.1.num_batches_tracked: copying from a non-
meta parameter in the checkpoint to a meta parameter in the current model, which
is a no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.1.conv1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)

```

```

    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.1.bn1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.1.bn1.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.1.bn1.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.1.bn1.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.1.bn1.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.1.conv2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.1.bn2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.1.bn2.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their

```

```

corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.1.bn2.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.1.bn2.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.1.bn2.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.1.conv3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.1.bn3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.1.bn3.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.1.bn3.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.1.bn3.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did

```

```

you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.1.bn3.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.2.conv1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.2.bn1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.2.bn1.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.2.bn1.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.2.bn1.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.2.bn1.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:

```

```

UserWarning: for layer3.2.conv2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.2.bn2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.2.bn2.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.2.bn2.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.2.bn2.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.2.bn2.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.2.conv3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.2.bn3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(

```

```

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.2.bn3.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.2.bn3.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.2.bn3.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.2.bn3.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.3.conv1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.3.bn1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.3.bn1.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.3.bn1.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)

```

```

    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.3.bn1.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.3.bn1.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.3.conv2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.3.bn2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.3.bn2.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.3.bn2.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.3.bn2.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.3.bn2.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state

```

```

dictionary to their corresponding key in the module instead of copying them in
place?)
    super().__load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.3.conv3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module.__load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.3.bn3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super().__load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.3.bn3.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super().__load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.3.bn3.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super().__load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.3.bn3.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super().__load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.3.bn3.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super().__load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.4.conv1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module.__load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.4.bn1.weight: copying from a non-meta parameter in the

```

```

checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.4.bn1.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.4.bn1.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.4.bn1.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.4.bn1.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.4.conv2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.4.bn2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.4.bn2.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:

```

```

UserWarning: for layer3.4.bn2.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.4.bn2.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.4.bn2.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.4.conv3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.4.bn3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.4.bn3.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.4.bn3.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.4.bn3.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(

```

```

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.4.bn3.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.5.conv1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.5.bn1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.5.bn1.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.5.bn1.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.5.bn1.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.5.bn1.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.5.conv2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their

```

```

corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.5.bn2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.5.bn2.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.5.bn2.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.5.bn2.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.5.bn2.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer3.5.conv3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.5.bn3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.5.bn3.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you

```

```

mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.5.bn3.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.5.bn3.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer3.5.bn3.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer4.0.conv1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.bn1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.bn1.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.bn1.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.bn1.running_var: copying from a non-meta parameter in

```

the checkpoint to a meta parameter in the current model, which is a no-op. (Did you mean to pass `assign=True` to assign items in the state dictionary to their corresponding key in the module instead of copying them in place?)

```
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.bn1.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
```

```
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer4.0.conv2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
```

```
module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.bn2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
```

```
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.bn2.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
```

```
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.bn2.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
```

```
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.bn2.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
```

```
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.bn2.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
```

```
super()._load_from_state_dict(
```

```

/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer4.0.conv3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.bn3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.bn3.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.bn3.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.bn3.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.bn3.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer4.0.downsample.0.weight: copying from a non-meta parameter
in the checkpoint to a meta parameter in the current model, which is a no-op.
(Did you mean to pass `assign=True` to assign items in the state dictionary to
their corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.downsample.1.weight: copying from a non-meta parameter
in the checkpoint to a meta parameter in the current model, which is a no-op.
(Did you mean to pass `assign=True` to assign items in the state dictionary to
their corresponding key in the module instead of copying them in place?)

```

```

    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.downsample.1.bias: copying from a non-meta parameter
in the checkpoint to a meta parameter in the current model, which is a no-op.
(Did you mean to pass `assign=True` to assign items in the state dictionary to
their corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.downsample.1.running_mean: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.downsample.1.running_var: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.0.downsample.1.num_batches_tracked: copying from a non-
meta parameter in the checkpoint to a meta parameter in the current model, which
is a no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer4.1.conv1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.1.bn1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.1.bn1.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.1.bn1.running_mean: copying from a non-meta parameter in

```

the checkpoint to a meta parameter in the current model, which is a no-op. (Did you mean to pass `assign=True` to assign items in the state dictionary to their corresponding key in the module instead of copying them in place?)

```
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.1.bn1.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.1.bn1.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer4.1.conv2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.1.bn2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.1.bn2.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.1.bn2.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.1.bn2.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
```

```

UserWarning: for layer4.1.bn2.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer4.1.conv3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.1.bn3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.1.bn3.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.1.bn3.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.1.bn3.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.1.bn3.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer4.2.conv1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)

```

```

    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.2.bn1.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.2.bn1.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.2.bn1.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.2.bn1.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.2.bn1.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer4.2.conv2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.2.bn2.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.2.bn2.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their

```

```

corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.2.bn2.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.2.bn2.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.2.bn2.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/module.py:2586:
UserWarning: for layer4.2.conv3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    module._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.2.bn3.weight: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.2.bn3.bias: copying from a non-meta parameter in the
checkpoint to a meta parameter in the current model, which is a no-op. (Did you
mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.2.bn3.running_mean: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did
you mean to pass `assign=True` to assign items in the state dictionary to their
corresponding key in the module instead of copying them in place?)
    super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.2.bn3.running_var: copying from a non-meta parameter in
the checkpoint to a meta parameter in the current model, which is a no-op. (Did

```

you mean to pass `assign=True` to assign items in the state dictionary to their corresponding key in the module instead of copying them in place?)

```
super()._load_from_state_dict(
/usr/local/lib/python3.12/dist-packages/torch/nn/modules/batchnorm.py:133:
UserWarning: for layer4.2.bn3.num_batches_tracked: copying from a non-meta
parameter in the checkpoint to a meta parameter in the current model, which is a
no-op. (Did you mean to pass `assign=True` to assign items in the state
dictionary to their corresponding key in the module instead of copying them in
place?)
```

```
super()._load_from_state_dict(
Loading weights:  0%|          | 0/530 [00:00<?, ?it/s]
```

DetrForObjectDetection LOAD REPORT from: facebook/detr-resnet-50

Key	
Status	
-----+-----	
-----+-----	
model.backbone.conv_encoder.model.layer4.0.downsample.1.num_batches_tracked	
UNEXPECTED	
model.backbone.conv_encoder.model.layer2.0.downsample.1.num_batches_tracked	
UNEXPECTED	
model.backbone.conv_encoder.model.layer1.0.downsample.1.num_batches_tracked	
UNEXPECTED	
model.backbone.conv_encoder.model.layer3.0.downsample.1.num_batches_tracked	
UNEXPECTED	

Notes:

- UNEXPECTED :can be ignored when loading from different task/architecture; not ok if you expect identical arch.

1 objectes:

99% cat